Autocomplete

answer every keystroke in a few milliseconds

Watch the 4-min video

In one line: precompute the best completions for every prefix offline (a **trie** with the **top-k stored at each node**, or an engine's **completion suggester** / **edge n-grams**), serve them from memory, and cut traffic with **edge caching** of popular prefixes and **client debouncing**.

1 Search under extreme time pressure

The user types i, p, h, and the box already suggests 'iphone 15 case'. Every keystroke is a request: a million people searching per minute, about 10 characters each, is roughly 10 million suggestion requests a minute (~170,000/s). Each answer must arrive in well under 100 ms end to end, or the suggestion shows up after the user has typed past it, so the server budget is a few milliseconds. Normal full-text search per keystroke would be too slow and too expensive.

2 The trie, with answers stored in it

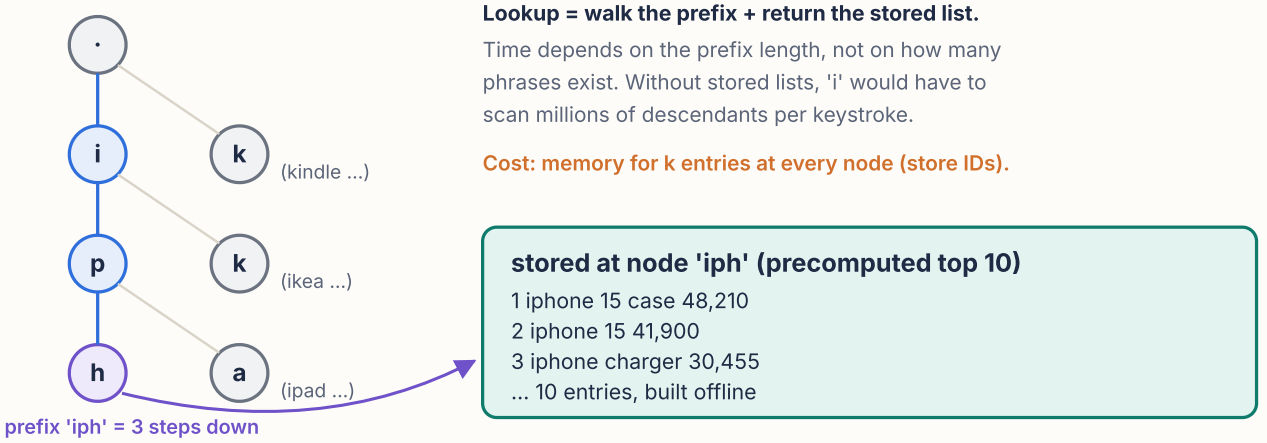


Figure 1. A trie (prefix tree). Each node is one character; each node also stores its top completions, so no subtree search is needed.

In a **trie**, each path from the root spells a prefix and every phrase below a node shares that prefix. Walking 'i' → 'p' → 'h' takes three steps whatever the dictionary size. But a short prefix like 'i' has millions of descendants, and ranking them per keystroke is far too slow. So the builder stores the **top k completions at every node**, computed bottom-up: a node's list is the best k among its own phrase and its children's lists. A lookup becomes 'walk the prefix, return the list'. Store phrase IDs, not strings, to keep memory down; compressed variants (radix trees, FSTs) merge single-child chains.

To scale, the whole structure is usually small enough to **replicate** to every server (millions of phrases → a few GB at most). If not, **shard by prefix** (a–f, g–m, ...), watching for hot first letters. Rebuild offline and ship an immutable snapshot that servers swap atomically, instead of updating a live trie under load.

3 Inside a search engine: edge n-grams or completion suggester

	Edge n-grams on a text field	Completion suggester	search_as_you_type field
How	index i, ip, iph, ipho... of each word; query the prefix as a normal term	in-memory FST per segment, built from a `completion` field	Elasticsearch field type with shingle and edge-n-gram sub-fields
Speed	a normal query: fast, but full query machinery	fastest: a pure prefix walk	normal query
Flexibility	any filters, scoring, mid-phrase words ('case' finds 'iphone case')	prefix of the input only; filters limited to predefined contexts (category, geo)	matches words in any position, phrase-prefix
Cost	bigger index (many terms per word)	heap memory; weights set at index time	several extra sub-fields

One rule for edge n-grams: use the n-gram analyzer at **index time only** and a plain analyzer at **search time**. Otherwise the query 'iph' is itself split into i, ip, iph and matches everything starting with 'i'.

4 Ranking suggestions

- **Popularity**: how often the phrase was searched (and, better, led to a click or purchase) over a window such as 30 days.
- **Recency**: time-decayed counts, plus a small streaming 'trending' layer (counts from the last hour) merged at query time, so breaking topics appear before the nightly rebuild.
- **Personalisation**: the user's own recent searches, merged on the client or in the service; this part can't be cached at the edge.
- **Hygiene**: blocklist offensive phrases, require a minimum count (so one person can't seed a suggestion), drop out-of-stock or zero-result phrases. Doing this offline keeps lookups cheap and policy reviewable.
- **Typos**: fuzzy prefix matching (edit distance 1 after the first couple of characters) rescues 'iphnoe'.

5 Making it cheap to serve

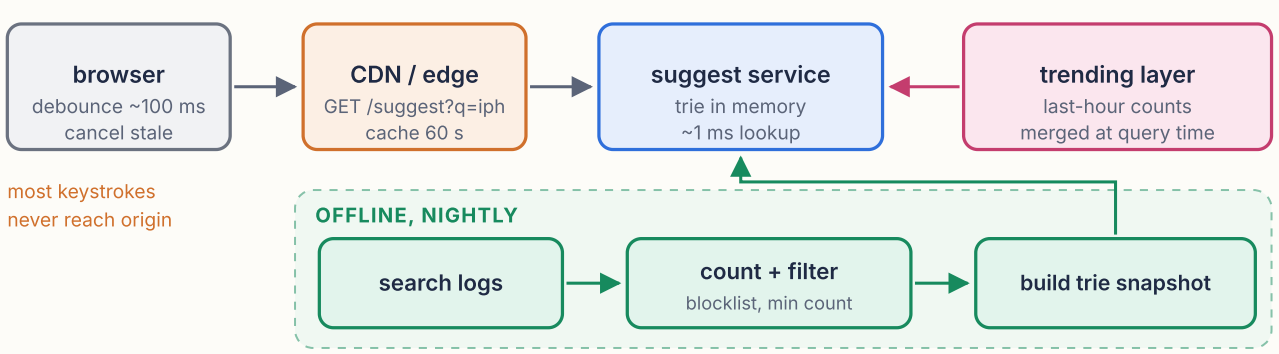


Figure 2. Serving path and offline build. The service only answers what the browser and the CDN didn't.

Prefix traffic is extremely skewed: everyone types 'i' and 'ip'. There are only 26 single-letter and 676 two-letter Latin prefixes, so those can be precomputed and cached at the **CDN** with a short TTL (e.g. 60 s), keyed by prefix and locale. On the client, **debounce** (send after ~100 ms without a keystroke), **cancel** in-

flight requests for prefixes the user typed past, and reuse the previous list when the new prefix just extends it and the list is still valid. Together these cut origin traffic by an order of magnitude or more.

WORKED EXAMPLE: SEARCH-BAR SUGGESTIONS

A nightly job reads 30 days of search logs, normalises phrases (lowercase, trim), counts them, drops phrases searched by fewer than 20 users or on the blocklist, and keeps the top few million. It builds a trie with the top 10 completions at every node and writes a snapshot. Each suggest server loads the snapshot into memory and answers `/suggest?q=iph` in about a millisecond. The CDN caches responses for a minute, so 'i', 'ip' and 'iph' hit the origin once a minute per edge location. The client waits until the user pauses about 100 ms. Result: suggestions feel instant, and the service handles a small fraction of the raw 170,000 keystrokes a second.

TRADE-OFFS

- **Precomputed top-k**: microsecond lookups, but more memory and only as fresh as the last build.
- **Edge n-grams**: filters and mid-word matches, but a larger index and slower than a dedicated structure.
- **CDN caching**: huge savings on short prefixes, but no personalisation in cached responses.
- **Longer debounce**: fewer requests, but suggestions feel laggy.

COMMON MISTAKES

- Running a full search query on every keystroke without debounce or caching.
- Applying the edge n-gram analyzer at search time too.
- Ranking by raw counts with no blocklist or minimum, so abuse becomes a suggestion.
- Showing results from a slow, stale request after a newer one already returned.

6 Key terms

Trie: prefix tree: one node per character

Edge n-gram: prefixes of a word indexed as terms

Debounce: wait for a pause before sending

Top-k per node: precomputed best completions for each prefix

Completion suggester: in-memory FST for fast prefix lookups

Trending layer: short-window counts merged at query time

7 Interview questions

Q1. How would you implement autocomplete at scale? BASIC

A trie or completion suggester with the top suggestions precomputed per prefix, ranked offline by popularity, held in memory, with caching for popular prefixes and debouncing on the client.

Q2. Edge n-grams or a completion suggester? BASIC

Edge n-grams are flexible and work with filters and mid-phrase words; completion suggesters are faster but limited to prefixes and predefined contexts.

Q3. A news event happens and the suggestion doesn't appear until tomorrow. Fix? SCENARIO

Add a streaming trending layer (counts over the last hour) merged with the nightly trie at query time, with its own hygiene filters.

Q4. How do you size the servers for 170,000 suggest requests/s? SCENARIO

Most short prefixes are served by the CDN and debounced clients; the remaining origin requests are ~1 ms in-memory lookups, so load-test one server, then divide the measured origin rate (after CDN hit rate) by its capacity with headroom.

Q5. Users complain that 'case' doesn't suggest 'iphone 15 case'. Why? SCENARIO

A prefix trie or completion suggester only matches from the start of the phrase; index word-level prefixes (edge n-grams or search_as_you_type) for infix matches.

Q6. Why store the top-k at every node instead of computing it? TRICKY

Short prefixes have millions of descendants; storing k IDs per node trades memory for constant-time lookups on exactly the prefixes that get the most traffic.

Q7. Can you cache personalised suggestions at the CDN? TRICKY

Not per user; cache the global list per prefix and locale at the edge, then merge the user's own history on the client or in a small follow-up call.

Next: SD-P-009 · Vector Search backendroadmap.com